On-the-Fly Temporal-Logic Task Allocation for Heterogeneous Multi-Robot Systems during Büchi Automaton Generation

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—This paper studies multi-robot task allocation under global linear temporal logic (LTL) specifications for heterogeneous robot teams. Existing methods typically construct the complete automaton before performing task allocation, which can delay the first feasible solution in large-scale problems and hinder rapid plan repair when robot availability changes during execution. To address these limitations, we propose an on-the-fly framework that performs temporal-logic task allocation within Büchi automaton generation. The method prunes infeasible and redundant transitions during generalized Büchi automaton construction, and then synchronously builds a nondeterministic Büchi automaton and a plan graph. The method prunes infeasible and redundant transitions during generalized Büchi automaton construction, and then synchronously builds a nondeterministic Büchi automaton and a plan graph. Once an accepting prefix-suffix structure is reached, the framework can immediately return an initial executable plan. After the plan graph is fully constructed, a feasible solution extracted from the completed graph is used to warm-start a branch-and-bound optimizer for further improvement. The same workflow is also used to repair the remaining mission when robots become unavailable during execution. Theoretical analysis establishes completeness of the proposed framework. The proposed framework is validated through extensive numerical simulations against strong baselines and hardware experiments. The results demonstrate that it generates feasible plans much faster in large-scale systems with complex LTL specifications, while also maintaining rapid plan repair when robots become unavailable during online execution.**

***Index Terms*—Linear temporal logic, multi-robot task allocation, heterogeneous robots, anytime search.**

I. INTRODUCTION

Multi-robot systems (MRS) have been widely used in applications such as persistent surveillance [1], logistics [2], and autonomous maintenance [3], where multiple robots can operate concurrently and cooperatively to improve efficiency and accomplish tasks that are beyond the capability of a single robot. In practice, many multi-robot tasks are subject to temporal requirements. For instance, in a smart factory or large warehouse, carriers may first transport tools or shelves to a target region, inspection robots may verify safety conditions before manipulation begins, and monitoring robots may be required to continuously patrol surrounding areas. Such task dependencies naturally introduce temporal constraints such as ordering, persistence, and synchronization. To specify these constraints in a rigorous way, many recent work have used formal languages such as linear temporal logic (LTL) [4]; see, for example, [5], [6].

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

In this work, we consider the multi-robot task allocation (MRTA) problem under temporal logic specifications, referred to as LTL-MRTA, which was formalized in [7]. In LTL-MRTA, the task is specified by an LTL formula, but the specific robots assigned to each subtask are not predetermined by the formula. For example, . For example, a specification may require two type-1 robots to complete a pickup-and-delivery task before waiting at the delivery site until a type-2 robot arrives, without prescribing which individual robots should execute these subtasks. This problem becomes increasingly difficult as the number of robots and the complexity of the task specification grow, since the associated search and optimization space may expand explosively. As a result, existing approaches, such as mixed-integer linear programming (MILP)-based methods [7], [8] and search-based methods [9], [10] can be computationally demanding in large-scale problems, even when only an initial feasible solution is required.

In practical robotic applications, however, it is often important to obtain an executable solution early, especially in large-scale settings where task execution needs to start promptly and decisions may need to be updated online. Planning-tree-based [6], [11] and partial-order-based methods [3] highlighted the practical importance of fast first-solution generation. However, planning-tree-based methods may be prone to local optima, while partial-order-based methods may be less effective for complex tasks due to the need to extract the partial-order relations among subtasks. Consequently, for LTL-MRTA, developing methods that can quickly return a feasible solution and further improve it toward optimality remains an important research direction. In real-robot execution, the available team may also change unexpectedly due to hardware faults, robot damage, localization failures, or battery depletion. Such events do not necessarily change the temporal-logic mission itself, but they invalidate assignments involving the unavailable robots and require the remaining mission to be repaired from the current execution state. Therefore, an LTL-MRTA planner should not only generate an executable plan before execution, but also support fast online replanning when the available robot set changes.

To address this problem, we propose a novel framework that integrates robot planning into the automaton generation process. Unlike existing methods that first construct the complete NBA and then perform planning on top of it, our approach introduces information about the robot team, the workspace, and intermediate planning results during the translation from LTL to automata. Specifically, we follow the translation pipeline of LTL2BA [12], a classical tool widely used in the literature [13], [14], which translates an LTL formula into a very weak alternating automaton (VWAA), then into a generalized Büchi

automaton (GBA), and finally into a nondeterministic Büchi automaton (NBA). During GBA construction, each candidate transition is screened before insertion against the robot types and cardinalities available in the team, and candidates whose simultaneous action requirements cannot be satisfied are discarded. After the GBA is constructed, transitions that impose unnecessary synchronization are removed. During NBA generation, each newly generated transition induces a subtask. A greedy task-allocation procedure is then applied to the induced subtask. If no feasible assignment exists, the corresponding plan-graph successor is discarded; otherwise, the feasible assignment is recorded in a shared plan graph. Consequently, an executable plan becomes available as soon as an accepting prefix–suffix structure is generated, before the complete NBA has been constructed. After the plan graph has been completed, the executable plan initializes the incumbent of a branch-and-bound optimizer, which explores alternative robot assignments over the generated graph to further reduce the makespan. When a robot becomes unavailable during execution, the method re-roots the retained plan graph at the last confirmed execution state and re-optimizes the unfinished assignments over the surviving team. If the retained graph contains no feasible accepting continuation for the surviving team, the current implementation reports infeasibility. In this way, the proposed method supports rapid first-solution generation as well as continued improvement toward better solutions.

A. Related Work

Existing planning methods under LTL constraints can generally be classified into bottom-up and top-down approaches. Bottom-up approaches usually formulate the problem as a task-coordination problem under local LTL specifications, where each robot is assigned a local LTL task and team behavior is achieved through communication, synchronization, and online replanning. For example, Guo et al. [15] studied a decentralized planning framework under local LTL specifications with online plan reconfiguration, while Yu et al. [16] developed a distributed motion-coordination strategy under local LTL specifications, where conflicts among neighboring robots are detected and resolved online to ensure safety and task satisfaction. In contrast, top-down approaches start from a global LTL formula that specifies the overall team mission. This paper focuses on the latter setting.

For top-down methods, a common strategy is to construct the product automaton of NBA translated from the global LTL formula and the discrete transition systems of the robots and then solve the resulting graph-search problem [9], [17]. However, this approach suffers from state explosion , because the number of states in the product automaton increases rapidly with the workspace/state-space size, the number of robots, and the complexity of the task, as reflected by the size of the automaton derived from the specification. To alleviate this issue, sampling-based methods [10], [18] incrementally construct trees that approximate the state space and transitions of the synchronous product automaton, instead of explicitly building the full product automaton. This significantly improves scalability while retaining probabilistic completeness and asymptotic optimality. However, such methods typically require the global LTL formula to explicitly assign subtasks to robots. When the robot assignment is not given a priori, a possible workaround is to enumerate all admissible robot combinations and connect them using OR operators, but this can result in exponentially long LTL formulas.

To avoid the combinatorial burden and reduced flexibility caused by explicitly allocate tasks to robots, some works decompose the global LTL specifications. For example, [14], [19] proposed the simultaneous task allocation and planning (STAP) framework, which constructs a team model that integrates task decomposition and task allocation. By decomposing the global task into parallelizable subtasks, the framework improves planning efficiency. However, these works do not explicitly address tightly coupled collaborative tasks requiring the simultaneous participation of multiple robots. Other works address global temporal-logic task planning by using mixed-integer linear programming (MILP) [7], [8], [20]. In particular, to mitigate the impact of workspace size on the computational burden, [7] proposed a hierarchical architecture that separates task planning from motion planning. Unfortunately, MILP-based methods can become computationally demanding in large-scale settings, even when only a first feasible solution is required.

Planning-decision-tree-based methods [11], [21] have been proposed to improve the speed of temporal-logic mission planning. These methods represents task progression and task allocation incrementally, enabling real-time planning for large-scale multi-robot systems. Although such methods are highly efficient in generating feasible solutions, they typically rely on greedy task-allocation decisions, and therefore can be prone to getting trapped in local optima. Liu et al. [3] analyze the task automaton to extract partial-order relations among subtasks, and then use a branch-and-bound search procedure to compute task assignments. Their method explicitly emphasizes the importance of obtaining a valid plan early: it is formulated as an anytime algorithm, so that a feasible solution can be returned quickly and then progressively improved as computation continues. However, when the task specification becomes more complex, the efficiency of obtaining the first feasible solution may decrease, since extracting the partial-order relations among subtasks can itself take additional time. As illustrated in the paper, for an automaton with hundreds of states, the algorithm may require tens or even hundreds of seconds to obtain the first partial order. Some work reduces the burden of complex specifications by changing or exploiting the structure of the task formula. Hierarchical temporal-logic specifications [22] organize complex specifications into a hierarchy. Liu et al. [23] exploit the case where an sc-LTL mission is given as a conjunc-tion of subformulas; each subformula is converted into a poset and the products of these posets are computed online to accommodate newly released tasks. These methods reduce the burden of complex specifications mainly by changing or exploiting the modeled structure of the LTL specification. While effective, this perspective may require the mission to be expressed in a suitable hierarchical, conjunctive, or otherwise structured form, and shift part of the complexity to specification design or task decomposition.

Overall, while existing top-down methods have advanced LTL-MRTA from different perspectives, they still struggle to simultaneously achieve rapid first-solution generation and effective subsequent optimization in large-scale problems. This limitation is particularly evident when substantial computation is required before a planning structure suitable for allocation can even be obtained. Instead of constructing the complete NBA first and then solving the allocation problem on top of it, the proposed method incorporates robot-team feasibility, workspace travel information, and partial planning results into the LTL-to-automaton translation process. During GBA construction, infeasible or decomposition-inducing transitions are pruned before they are propagated to the NBA stage. During the NBA construction stage, the NBA and a shared directed planning graph are generated synchronously. Whenever a new automaton transition becomes available, the corresponding transition is evaluated by a greedy executable assignment, and newly discovered successor Buchi states are ordered within the current pending batch by a lightweight residual-obligation score. Once an accepting prefix-suffix structure is reached, an executable initial plan can be extracted and used as a warm start for the subsequent branch-and-bound assignment optimizer. When robot failures occur during execution, the proposed method can rapidly repair the plan for the remaining mission. The proposed framework supports both fast initial planning and continued improvement toward higher-quality solutions.

B. Contributions

We propose a new on-the-fly approach to LTL-MRTA that performs task allocation as the NBA is generated. Unlike conventional methods that construct the complete NBA before allocation, our method performs task allocation incrementally using a greedy procedure during NBA construction. During this process, feasible partial assignments and their execution states are recorded in a plan graph. This graph grows synchronously with the NBA. An executable plan becomes available as soon as an accepting structure is reached, even if the NBA is not yet complete. One innovation of our approach is that the GBA is pruned before NBA construction begins. We discard transitions that are infeasible for the available robot team or impose avoidable synchronization. This reduces the number of transitions passed to the NBA stage. Another distinctive aspect is that a residual-obligation score guides both the NBA and the plan graph as they are constructed. The score reflects the current plan cost and the estimated travel needed for the remaining tasks. Rather than following a DFS order used in standard LTL2BA construction, our method gives priority to Büchi successors with lower scores. The NBA and the plan graph are constructed together. Thus, the same ordering also guides the exploration of task-allocation branches. More promising branches are explored earlier. This helps the method obtain a high-quality initial plan sooner. To the best of our knowledge, this is the first LTL-MRTA framework to perform task allocation during score-guided NBA construction.

The rest of the article is organized as follows. Sections II and III present the preliminaries and formulate the LTL-MRTA problem, respectively. Section IV provides an overview of the proposed framework. Section V introduces the GBA-level pruning, and Section VI details the construction of the NBA and the plan graph. Section VII presents the branch-and-bound optimization, while Section VIII describes plan adaptation under robot unavailability. Sections IX and X provide the complexity and theoretical analyses, respectively. Section XI and XII report the numerical Aand physical experiments, and Section XIII concludes the article.

II. PRELIMINARIES

A. Linear Temporal Logic

Linear temporal logic (LTL) is widely used to specify temporal tasks over a set of atomic propositions AP. Following the standard syntax, an LTL formula over AP is recursively defined by

$$\varphi := \top \mid \pi \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \bigcirc\varphi \mid \varphi_1 \,\mathcal{U}\, \varphi_2,$$

where $\pi \in AP$, $\top$ denotes the constant true, $\bigcirc$ is the next operator, and $\mathcal{U}$ is the until operator. Other temporal operators can be derived in the standard way, including eventually $\Diamond\varphi := \top\,\mathcal{U}\,\varphi$, always $\Box\varphi := \neg\Diamond\neg\varphi$.

Let $\Sigma = 2^{AP}$ be the alphabet. An infinite word over Σ is a sequence $w = \sigma_0\sigma_1\cdots \in \Sigma^\omega$, where each $\sigma_k \in \Sigma$ denotes the set of atomic propositions satisfied at step k. The notation $w \models \varphi$ means that the word w satisfies the LTL formula φ. The language of φ is defined as $\mathrm{Words}(\varphi) := \{\, w \in \Sigma^\omega \mid w \models \varphi \,\}$.

It is well known that any satisfiable LTL formula can be translated into a nondeterministic Büchi automaton (NBA).

Definition 1 (Nondeterministic Büchi Automaton)**.** A nondeterministic Büchi automaton (NBA) is a tuple

$$\mathcal{B}_\varphi = (Q_B, Q_{B,0}, \Sigma_B, \rightarrow_B, Q_{B,F}), \tag{1}$$

where Q_B is a finite set of states, $Q_{B,0} \subseteq Q_B$ is the set of initial states, Σ_B is the alphabet, $\rightarrow_B \subseteq Q_B \times \Sigma_B \times Q_B$ is the transition relation, and $Q_{B,F} \subseteq Q_B$ is the set of accepting states.

An infinite run of $\mathcal{B}$ over a word $w = \sigma_0\sigma_1\cdots$ is a sequence $\rho = q_0q_1\cdots$ such that $q_0 \in Q_{B,0}$ and $(q_k, \sigma_k, q_{k+1}) \in \rightarrow_B$ for all $k \geq 0$, where each $\sigma_k \in \Sigma_B$. The run is accepting if it visits accepting states infinitely often, i.e., $\mathrm{Inf}(\rho) \cap Q_{B,F} \neq \emptyset$. For a satisfiable LTL formula, an accepting run can be written in the prefix–suffix form, where the prefix reaches an accepting state once and the suffix repeats a cycle around that accepting state indefinitely.

B. LTL2BA Translation

In this work, we follow the translation pipeline of LTL2BA [12] to construct automata from LTL specifications. Given an LTL formula φ, LTL2BA first rewrites φ into negative normal form (NNF), so that negation only appears in front of atomic propositions. It then applies rewriting rules to simplify the formula and reduce the number of temporal operators before automaton generation. An abstract syntax tree is then generated from the simplified formula.

Definition 2 (Abstract Syntax Tree)**.** Given an LTL formula φ in negative normal form, its abstract syntax tree (AST), denoted by $\mathcal{T}_\varphi$, is a rooted ordered tree in which each internal node is labeled by a logical or temporal operator, and each leaf is labeled by an atomic proposition or its negation. The recursive structure of $\mathcal{T}_\varphi$ represents the syntactic decomposition of φ and provides the basis for extracting temporal subformulas during automaton construction.

Starting from the AST, LTL2BA first constructs a very weak alternating automaton (VWAA). An alternating automaton may require multiple successor states to hold simultaneously under the same input symbol. Moreover, the "very weak" property imposes a partial order on states, so that transitions can only remain at the same level or move downward in this order, which yields a simpler automaton structure.

Definition 3 (Very Weak Alternating Automaton)**.** A very weak alternating automaton (VWAA) is a tuple

$$\mathcal{V}_\varphi = (Q_v, \Sigma, \delta_v, I_v, F_v, \preceq_v),$$

where Q_v is a finite set of states, Σ is the input alphabet, δ_v is the transition function, I_v is the initial condition, $F_v \subseteq Q_v$ is the set of accepting states, and $\preceq_v$ is a partial order on Q_v such that every state appearing in the transition of a state $q_v \in Q_v$ is lower than or equal to q_v under $\preceq_v$.

Based on the VWAA, LTL2BA then constructs a generalized Büchi automaton (GBA), which preserves multiple acceptance requirements in a compact intermediate form before the final degeneralization step.

Definition 4 (Generalized Büchi Automaton)**.** A generalized Büchi automaton (GBA) is a tuple

$$\mathcal{G}_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, \mathcal{F}_g),$$

where Q_g is a finite set of states, $Q_{g,0} \subseteq Q_g$ is the set of initial states, Σ is the input alphabet, $\rightarrow_g \subseteq Q_g \times \Sigma \times Q_g$ is the transition relation, and $\mathcal{F}_g = \{F_{g,1}, \ldots, F_{g,m}\}$ is a finite collection of accepting transition sets with $F_{g,i} \subseteq \rightarrow_g$ for all $i \in \{1, \ldots, m\}$. A run is accepting if, for every $F_{g,i} \in \mathcal{F}_g$, it traverses transitions in $F_{g,i}$ infinitely often.

The VWAA-to-GBA construction is briefly summarized here. Following [12], each GBA state $q_g \in Q_g \subseteq 2^{Q_v}$ is a set of VWAA states and is identified with their conjunction. For example, $q_g = \{q_{v,1}, \ldots, q_{v,n}\} \equiv q_{v,1} \wedge \cdots \wedge q_{v,n}$, which means that all states in q_g must hold simultaneously. To generate an outgoing transition from q_g, one transition is selected from each $\delta_v(q_{v,i})$. The selected transition conditions are conjoined, or equivalently, their sets of satisfying input symbols are intersected. Their successor-state sets are also combined to form a successor GBA state q'_g. Each consistent combination produces a candidate transition $e_g = (q_g, \sigma, q'_g)$, where σ denotes the combined transition condition. After the standard implication-based simplification inherited from LTL2BA, the retained candidate transitions are inserted into $\rightarrow_g$.

The co-Büchi acceptance condition of the VWAA is transferred to the transition-based acceptance condition of the GBA. In particular, each co-Büchi state $f \in F_v$ induces one accepting transition set in $\mathcal{F}_g$. This acceptance information is subsequently used during the GBA-to-NBA transformation. After constructing the GBA, LTL2BA applies a standard degeneralization step to obtain the final nondeterministic Büchi automaton (NBA), denoted by $\mathcal{B}_\varphi$. Therefore, the overall translation pipeline can be summarized as $\varphi \rightarrow \mathcal{T}_\varphi \rightarrow \mathcal{V}_\varphi \rightarrow \mathcal{G}_\varphi \rightarrow \mathcal{B}_\varphi$.

A key advantage of LTL2BA is that simplifications are performed throughout the translation process. Besides simplifying the formula before automaton generation, the algorithm also simplifies intermediate automata on-the-fly by removing inaccessible states, eliminating implied transitions, and merging equivalent states whenever possible. These operations reduce the number of intermediate states and transitions and improve both time and memory efficiency. In this paper, we build on this translation pipeline and further incorporate planning-related information during automaton construction so that the generated automaton structure is better aligned with the downstream task-planning problem.

III. PROBLEM DESCRIPTION

A. Workspace and Heterogeneous Robot Team

Consider a bounded workspace $\mathcal{W}$ containing a finite set of mutually disjoint regions of interest $\mathcal{L} = \{l_1, l_2, \ldots, l_{n_l}\}$. let $\mathcal{O} = \{o_1, o_2, \ldots, o_{n_o}\}$ denote the non-interested regions such as obstacles and forbidden regions. We assume that regions of interest do not overlap with each other and regions of non-interest , i.e., $l_i \cap o_j = \emptyset, \quad \forall i \in \{1, \ldots, n_l\}, \ \forall j \in \{1, \ldots, n_o\}, \ l_i \cap l_j = \emptyset, \quad \forall i \neq j$.

A team of heterogeneous robots is denoted by $\mathcal{R} = \{r_1, r_2, \ldots, r_{n_r}\}$. The robots belong to a finite set of types $\mathcal{T} = \{1, 2, \ldots, n_t\}$, where each type represents a specific capability, such as sensing, manipulation, or mobility. Each robot belongs to exactly one type. Let $\tau(r_k) \in \mathcal{T}$ denote the type of robot r_k, and let $\mathcal{R}_j = \{r_k \in \mathcal{R} \mid \tau(r_k) = j\}$ be the set of robots of type j. The position of robot r_k is denoted by $x(r_k) \in \mathcal{W}$, and its constant travel speed is denoted by $v(r_k) > 0$. For each robot $r_k \in \mathcal{R}$, its motion between regions is modeled by its own transition graph $\mathcal{G}_{r,k} = (V_{r,k}, \rightarrow_{r,k})$, $V_{r,k} \subseteq \mathcal{L}$ is the set of regions accessible to robot r_k and $\rightarrow_{r,k} \subseteq V_{r,k} \times V_{r,k}$ denotes the set of region-to-region transitions permitted for robot r_k. For any position $x \in \mathcal{W}$ and target region $l_i \in \mathcal{L}$, let $d_k(x, l_i)$ denote the shortest feasible distance that robot r_k must travel from x to reach l_i. In particular, $d_k(x(r_k), l_i)$ is the distance from the current position of r_k to l_i. We set $d_k(x, l_i) = +\infty$ if l_i is unreachable by r_k.

B. Task Specification

The temporal-logic specification is defined over a set of atomic propositions AP. Before formulating the temporal logic task, we introduce two classes of atomic propositions. (i) $\mathcal{P} \triangleq \{p_i \mid l_i \in \mathcal{L}\}$ contains region propositions, where p_i is true when at least one available robot is located at region l_i. (ii) Let a_i be an action proposition. For example, a_i can represent a group of robots visits a specified target

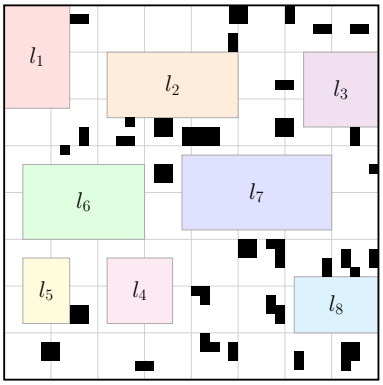

Fig. 1. Workspace of experiments.

region. Let $\mathcal{A} \triangleq \{a_1, a_2, \ldots, a_{n_q}\}$ denote the set of all action propositions. Each $a_i \in \mathcal{A}$ is associated with an execution requirement $c(a_i) = (\tau_i, n_i, l_i)$, where $\tau_i \in \mathcal{T}$ is the required robot type, $n_i \in \mathbb{N}$ is the required number of robots of type τ_i, and $l_i \in \mathcal{L}$ is the target region. The proposition a_i is true, i.e., $a_i = \top$, if and only if the corresponding action is performed by n_i robots of type τ_i at region l_i. In the factory scenario, an action proposition can represent a region-level operation such as inspection, material delivery, collaborative repair, or monitoring. The tuple $c(a)$ specifies the required robot capability, the number of robots that must participate, and the region where the operation must be completed. The complete set of atomic propositions used in the temporal logic specification is defined as $\mathcal{AP} \triangleq \mathcal{P} \cup \mathcal{A}$.

Example 1. For example, as shown in Fig.**??** consider a large warehouse facility with eight regions of interest, denoted by $\mathcal{L} = \{l_1, l_2, \ldots, l_8\}$. l_1 and l_2 are inbound receiving and temporary storage areas, l_3 and l_4 are order-picking and outbound loading areas, l_5 and l_6 are inspection and equipment-maintenance areas, and l_7 and l_8 are dispatch areas. The robot team consists of five robots, $\mathcal{R} = \{r_1, r_2, r_3, r_4, r_5\}$, where r_1, r_2, r_3 are type-1 robots and r_4, r_5 are type-2 robots. Consider a safety-inspection task $\Diamond\big(a_1 \wedge \Diamond(a_2 \wedge (a_3))\big) \wedge \Box\neg p_8$. Here, $c(a_1) = (2, 2, l_5), c(a_2) = (1, 1, l_6)$ and $c(a_3) = (1, 1, l_1)$. This formula requires all robots to avoid the restricted dispatch area l_8 throughout execution. It also requires two type-2 robots to first perform a collaborative safety inspection at l_5, after which one type-1 robot checks the equipment-maintenance area l_6 and another type-1 robot inspects the inbound receiving area l_1.

C. Problem Definition

Consider an LTL formula φ over $\mathcal{AP}$. Let $B_\varphi = (Q_B, Q_{B,0}, \Sigma_B, \rightarrow_B, Q_{B,F})$ be the NBA associated with φ. Here, Q_B is the finite set of Buchi states, $Q_{B,0}$ is the set of initial states, $\Sigma_B = 2^{AP}$ is the alphabet, $\rightarrow_B \subseteq Q_B \times \Sigma_B \times Q_B$ is the transition relation, and $Q_{B,F}$ is the set of accepting states. To connect the generated NBA structure with task-level planning, we associate each NBA transition with a subtask.

Definition 5 (Subtask)**.** Consider an NBA transition $(q_i, \sigma_i, q_j) \in \rightarrow_B$, where $q_i, q_j \in Q_B$ are NBA states. Let σ_i^{s} denote the self-loop condition at NBA state q_i. If no holding condition is required, we set $\sigma_i^{\mathrm{s}} = \top$. The subtask induced by this transition is defined as below.

$$\omega_i = (\sigma_i, \sigma_i^{\mathrm{s}}). \tag{2}$$

Here, σ_i specifies the propositions that enable the transition from q_i to q_j, while σ_i^{s} specifies the propositions that must be maintained before this transition is completed.

A plan of φ is defined as $\Pi = (\boldsymbol{\omega}, \boldsymbol{\rho}, \boldsymbol{\mathcal{C}})$ where $\boldsymbol{\omega} = \omega_0\omega_1\omega_2\cdots$ is the corresponding subtask sequence, $\boldsymbol{\rho} = q_0q_1q_2\cdots$ is the corresponding sequence of NBA states, $\boldsymbol{\mathcal{C}} = \mathcal{C}_0\mathcal{C}_1\mathcal{C}_2\cdots$ is the corresponding robot-assignment sequence. Specifically, for subtask ω_k, let $\mathcal{A}(\omega_k) \subseteq \mathcal{A}$ denote the set of action propositions that must be true at this transition. The assignment $\mathcal{C}_k : \mathcal{A}(\omega_k) \rightarrow 2^{\mathcal{R}}$ maps each required action proposition to the set of robots selected to execute it. Assignments for different action propositions in the same ω_k must be disjoint, i.e., $\mathcal{C}_k(a_i) \cap \mathcal{C}_k(a_j) = \emptyset, \quad \forall a_i \neq a_j,\ a_i, a_j \in \mathcal{A}(\omega_k)$.

The accepting run of NBA can be written in a prefix–suffix structure. Thus, $\boldsymbol{\rho}$ can be written as $\boldsymbol{\rho} = \boldsymbol{\rho}_{\mathrm{pre}}\big(\boldsymbol{\rho}_{\mathrm{suf}}\big)^{\omega}$, where $\boldsymbol{\rho}_{\mathrm{pre}}$ is the prefix run and $\boldsymbol{\rho}_{\mathrm{suf}}$ is the suffix run. The corresponding plan Π admits the same prefix–suffix structure, i.e. $\Pi = \Pi_{\mathrm{pre}}(\Pi_{\mathrm{suf}})^{\omega}$. Here, Π_{pre} is the plan segment associated with $\boldsymbol{\rho}_{\mathrm{pre}}$, and Π_{suf} is the finite plan segment associated with one traversal of the suffix cycle $\boldsymbol{\rho}_{\mathrm{suf}}$. To represent one completed execution of the prefix–suffix plan, we define the finite plan $\Pi_{\mathrm{fin}} = \Pi_{\mathrm{pre}}\Pi_{\mathrm{suf}}$. Let $T(\Pi_{\mathrm{fin}})$ denote the completion time of Π_{fin}. The cost is defined as below.

$$J_\varphi = T(\Pi_{\mathrm{fin}}). \tag{3}$$

Problem III.1. Consider a workspace $\mathcal{W}$ (shown in Figure **??**) and a heterogeneous robot team $\mathcal{R}$ with type set $\mathcal{T}$, and a LTL formula φ defined over the atomic proposition set $\mathcal{AP} = \mathcal{P} \cup \mathcal{A}$. The goal is to develop a task plan for each robot such that the specification φ is satisfied and the cost in Eq. 3 is minimized.

IV. System Overview

This section presents the proposed framework for LTL-MRTA, as summarized in Algorithm 1. The key idea is to integrate task-allocation reasoning into the LTL-to-automaton translation process, instead of constructing the complete NBA before planning. Specifically, the input LTL formula is first parsed into an abstract syntax tree and then translated into a VWAA following the LTL2BA pipeline. During the subsequent GBA construction, transitions that cannot be realized by the available robot team are pruned immediately. After the GBA is generated, decomposition-inducing transitions are further removed to avoid unnecessary synchronization among independent subtasks.

Given the pruned GBA, the framework constructs the NBA and a shared directed plan graph synchronously. Each newly generated NBA transition induces a subtask, which is evaluated on the fly using the robot assignments, predicted positions, and completion times stored in the current plan node. Infeasible successors are discarded, whereas feasible

task progressions are inserted into the plan graph. Once an accepting prefix–suffix structure is reached, an initial feasible plan is extracted. After the plan graph is completed, this feasible plan is used to warm-start a branch-and-bound optimizer, which further improves the completion time over the generated plan graph.

Algorithm 1 On-the-fly Task Allocation framework

Require: LTL formula φ, workspace $\mathcal{W}$, robot team $\mathcal{R}$, atomic propositions $\mathcal{AP}$.
Ensure: Feasible plan $\Pi^\star$.
1: Parse φ and construct the VWAA $\mathcal{V}_\varphi$.
2: Construct and prune the GBA $\mathcal{G}_\varphi^-$.
3: Initialize the plan graph G_P with the root plan node ν_0.
4: $(G_P, \Pi_{\text{first}}) \leftarrow$ PlanGraph_Construct($\mathcal{G}_\varphi, \mathcal{R}, \nu_0$).
5: $\Pi^\star \leftarrow$ BnB_Optimizer(G_P, Π_{first}).
6: **return** $\Pi^\star$.

V. Pruning for GBA

In this section, we introduce the proposed pruning procedure during the construction of GBA. Unlike methods that perform pruning after the complete NBA has been constructed, the proposed method prunes the GBA both during and after the VWAA-to-GBA construction stage. This early pruning reduces the size of the GBA and prevents unnecessary transitions from being propagated to the subsequent NBA construction and planning stages. The proposed GBA pruning consists of the following two parts.

1) Infeasible Transition Pruning During GBA Construction: The simplifications performed during the construction of GBA rely on the logical formula and the evolving automaton structure, without incorporating information about the robot team or the workspace. We therefore introduce a robot-team feasibility check during GBA construction. Before a candidate transition $e_g = (q_g, \sigma_g, q_g')$ is inserted into $\rightarrow_g$, the action requirements in σ_g are checked against the available robot team $\mathcal{R}$. If no subset of $\mathcal{R}$ can satisfy the requirements, e_g is infeasible for the considered system and is discarded immediately. Otherwise, it is retained for subsequent GBA construction. For example, suppose that a candidate transition requires three type-1 robots to perform an action at region l_1 and another three type-1 robots to perform an action at region l_2 simultaneously. If $\mathcal{R}$ contains only five type-1 robots, no feasible robot assignment can satisfy the transition condition. The candidate transition is therefore discarded before it is inserted into $\rightarrow_g$.

2) Pruning of Progress-Consistent Decomposable Transitions: The second pruning step is performed after the GBA has been constructed. Although pruning is applied at the GBA level, it targets unnecessary synchronization that would otherwise be inherited by the resulting NBA. Since each NBA transition induces a subtask, requiring several independent subtasks to be satisfied at the same transition can be unnecessarily restrictive [3]. The main challenge is to identify and remove such synchronization before the complete NBA is available. A GBA transition can therefore be removed only when decomposing its combined task requirements preserves the acceptance progress used in the subsequent GBA-to-NBA degeneralization. To formalize this decomposition without restricting it to a particular automaton type, we first introduce a general definition of transition decomposability. The definition applies to both NBA and GBA transitions.

Definition 6 (Decomposable transition)**.** Consider an automaton with state set Q, alphabet Σ, and transition relation $\rightarrow \subseteq Q \times \Sigma \times Q$. A transition $e_{ij} = (q_i, \sigma_{ij}, q_j) \in \rightarrow$ is called decomposable if there exist a state $q_k \in Q$ and two transitions $(q_i, \sigma_{ik}, q_k) \in \rightarrow$ and $(q_k, \sigma_{kj}, q_j) \in \rightarrow$ such that $\sigma_{ij} = \sigma_{ik} \cup \sigma_{kj}$.

Definition 6 applies to both NBA and GBA transitions. For GBA pruning, however, decomposability alone is not sufficient because degeneralization also tracks progress through multiple accepting transition sets. We therefore call a decomposable GBA transition progress-consistent if its decomposition produces the same acceptance-progress update as the original transition for every degeneralization index. Only progress-consistent decomposable transitions are eligible for pruning. Let $m_g := |\mathcal{F}_g|$ and $\mathcal{F}_g = \{F_{g,1}, \ldots, F_{g,m_g}\}$. For a GBA transition $e^g \in \rightarrow_g$, define its acceptance-set index set as $\mathrm{Acc}(e^g) := \{h \in \{1, \ldots, m_g\} \mid e^g \in F_{g,h}\}$. Each state of the degeneralized NBA is represented by (q_g, ℓ), where $\ell \in \mathcal{I}_F := \{0, \ldots, m_g\}$ is the current acceptance-progress index. Let $\eta(\ell, e^g)$ denote the progress obtained after traversing e^g from progress ℓ. Set $\bar{\ell} = 0$ if $\ell = m_g$ and $\bar{\ell} = \ell$ otherwise. Then $\eta(\ell, e^g) := \max\{r \in \{\bar{\ell}, \ldots, m_g\} \mid \{\bar{\ell}+1, \ldots, r\} \subseteq \mathrm{Acc}(e^g)\}$. This update provides a direct test of progress consistency. Given an initial progress ℓ, the candidate decomposition first computes $\eta(\ell, e_{ik}^g)$ and passes the returned index to e_{kj}^g to obtain $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g)$. This result is compared with $\eta(\ell, e_{ij}^g)$, and the decomposition is accepted when the two updates agree for all admissible initial progress indices. The following definition formalizes this criterion.

Definition 7 (Progress-consistent decomposable transition)**.** A decomposable GBA transition $e_{ij}^g \in \rightarrow_g$ is called progress-consistent decomposable if it admits a decomposition (e_{ik}^g, e_{kj}^g) according to Definition 6 such that $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g) = \eta(\ell, e_{ij}^g)$ for every $\ell \in \mathcal{I}_F$. Such a pair (e_{ik}^g, e_{kj}^g) is called a progress-consistent decomposition witness of e_{ij}^g.

Lemma 1. *Every NBA transition induced by a progress-consistent decomposable GBA transition is decomposable.*

Proof. Let $e_{ij}^g = (q_{g,i}, \sigma_{ij}, q_{g,j})$ be a progress-consistent decomposable GBA transition, and let (e_{ik}^g, e_{kj}^g) be one of its progress-consistent decomposition transitions, where $e_{ik}^g = (q_{g,i}, \sigma_{ik}, q_{g,k})$ and $e_{kj}^g = (q_{g,k}, \sigma_{kj}, q_{g,j})$. Fix an arbitrary acceptance-progress index $\ell \in \mathcal{I}_F$. The transition e_{ij}^g induces the NBA transition $((q_{g,i}, \ell), \sigma_{ij}, (q_{g,j}, \eta(\ell, e_{ij}^g)))$. The transition induces successive NBA transitions through $(q_{g,k}, \eta(\ell, e_{ik}^g))$, with terminal state $(q_{g,j}, \eta(\eta(\ell, e_{ik}^g), e_{kj}^g))$. By progress consistency, $\eta(\eta(\ell, e_{ik}^g), e_{kj}^g) = \eta(\ell, e_{ij}^g)$. Thus, both representations reach the same terminal NBA state. Moreover, $\sigma_{ij} = \sigma_{ik} \cup \sigma_{kj}$ follows from Definition 6. Therefore, the NBA transition induced by e_{ij}^g is decomposable. Since ℓ

is arbitrary, the result holds for every NBA transition induced by e_{ij}^g. □

Lemma 2. *If a feasible solution can be generated from the original GBA, then an equivalent feasible solution can also be generated from the pruned GBA.*

Proof. Let $\mathcal{B}_\varphi$ and $\mathcal{B}_\varphi^-$ be the NBAs generated from the original and pruned GBAs, respectively. Consider a feasible solution Π represented by an accepting run ρ of $\mathcal{B}_\varphi$. A GBA transition removed by infeasible-transition pruning cannot induce a transition used by Π. Otherwise, Π would contain a task requirement that cannot be satisfied by the robot team, which contradicts its feasibility. Now consider a progress-consistent decomposable GBA transition removed during the second pruning step. By Lemma 1, every NBA transition induced by this GBA transition is decomposable. The corresponding decomposition reaches the same terminal NBA state and preserves the acceptance progress. Moreover, its transition labels jointly reproduce the original transition label. Thus, the decomposition introduces no additional task requirements. Applying the decomposition to every affected transition in ρ produces an accepting run of $\mathcal{B}_\varphi^-$. If a transition involved in a decomposition is also pruned, its own decomposition is applied in the pruning order. Since the GBA contains finitely many transitions, this decomposition process terminates with transitions retained in the pruned GBA. The resulting accepting run represents a feasible solution equivalent to Π. □

VI. On-the-Fly Plan Graph Construction

After GBA-level pruning, the NBA and the plan graph are constructed synchronously. As the NBA is generated, each candidate transition induces a subtask. The greedy allocation procedure evaluates this subtask from the current plan nodes. A successful allocation extends the plan graph and updates the predicted execution state of the robot team. Once an accepting prefix–suffix structure is reached, an executable plan can be extracted without constructing the complete NBA. To guide this joint construction, a residual-obligation score is used to order newly generated NBA states. In contrast to the original LTL2BA construction, which follows the DFS generation order, the proposed method gives priority to states associated with more promising partial plans. Since the plan graph grows together with the NBA, this ordering also guides the exploration of task-allocation branches and helps obtain a high-quality initial plan earlier.

A. Plan Graph and Plan Node

During NBA construction, feasible task allocations are incrementally recorded in a plan graph. Each plan node associates an NBA state with the corresponding robot assignment and predicted execution state. A plan node is formally defined as follows.

Definition 8 (Plan node)**.** Consider the plan graph $G_P = (V_P, E_P)$, where V_P is the set of plan nodes and $E_P \subseteq V_P \times V_P$ is the set of directed edges. Each plan node is denoted by ν_i, and $V_P = \{\nu_i \mid i = 0, 1, \ldots, N_P\}$, where ν_0 is the root node and N_P+1 is the number of nodes in the generated plan graph. Each plan node $\nu_i \in V_P$ is defined as

$$\nu_i = (q_{B,i}, \mathrm{Prog}_i, \omega_i, \mathcal{C}_i, X_i, \Theta_i).$$

Here, $q_{B,i} \in Q_B$ is the NBA state associated with ν_i, and $\mathrm{Prog}_i \in \{\mathrm{pre}, \mathrm{suf}, \mathrm{acc}, \mathrm{dead}\}$ denotes the planning status of ν_i. Specifically, $\mathrm{Prog}_i = \mathrm{pre}$ indicates that the current node is in the prefix stage and $\mathrm{Prog}_i = \mathrm{suf}$ indicates that the current node is in the suffix stage. $\mathrm{Prog}_i = \mathrm{acc}$ indicates that the node completes an accepting prefix–suffix structure, while $\mathrm{Prog}_i = \mathrm{dead}$ indicates that the candidate is discarded and will not be further expanded. For each non-root node ν_i, $\mathrm{par}(\nu_i)$ denotes the predecessor used to compute its stored execution state and to extract the plan. ω_i is the subtask associated with ν_i, and $\mathcal{C}_i$ is the corresponding robot assignment. The vector $X_i = (x_i(r_1), x_i(r_2), \ldots, x_i(r_{n_r}))$ denotes the predicted robot positions, where $x_i(r_k)$ is the predicted position of robot r_k at plan node ν_i. The vector $\Theta_i = (\theta_i(r_1), \theta_i(r_2), \ldots, \theta_i(r_{n_r}))$ denotes the predicted completion times, where $\theta_i(r_k)$ is the predicted completion time of robot r_k.

By Definition 8, a plan node records the progress in the automaton and the predicted execution state of the robot team. Specifically, $q_{B,i}$, Prog_i, and ω_i describe where the current partial plan is in the NBA and which subtask is being considered. The assignment $\mathcal{C}_i$ specifies which robots execute the current subtask, while X_i and Θ_i summarize the predicted robot configuration after this subtask is completed.

B. Construction of Plan Graph

The plan graph is constructed synchronously with the NBA. The root node ν_0 is associated with the initial NBA state $q_{B,0}$ and the initial team configuration. Its subtask and robot-assignment fields are empty. For every $r_k \in \mathcal{R}$, its predicted position and completion time are initialized as $x_0(r_k) = x(r_k)$ and $\theta_0(r_k) = 0$, respectively. For a Buchi state q_B, there may exist multiple active plan-node anchors attached to it. These anchors correspond to different partial execution contexts reaching the same logical automaton state. For an outgoing transition $e_B : q_B \xrightarrow{\sigma} q_B'$ and a parent anchor ν_i attached to q_B, the induced subtask ω_{ij} is evaluated at the planning level. The planner first checks whether a compatible successor anchor already exists at q_B'. Compatibility is determined by the destination Buchi state, the prefix-suffix progress flag, and the positive/ negative transition-label signature. If such an anchor exists, the graph adds a structural edge from ν_i to that anchor. The successor runtime payload is recomputed under the new parent context, and its representative predecessor is updated only if the new connection yields a strictly smaller realized cost. The accepted improvement is then propagated to downstream shared successors. If no compatible anchor is available, a new plan node is generated. The executable payload of a newly generated successor is obtained by a greedy synchronized assignment. For each required action proposition $a \in A(\omega_{ij})$, let

$$c(a) = (\tau_a, n_a, l_a) \tag{4}$$

Algorithm 2 On-the-fly NBA and plan-graph construction

Require: Pruned GBA $\mathcal{G}_{\varphi}^{-}$, robot team $\mathcal{R}$, and initial plan node ν_0.
Ensure: Plan graph G_P and first feasible plan Π_{first}.
Initialize $G_P = (V_P, E_P)$ with root node $\nu_0 = (q_{B,0}, \text{pre}, \emptyset, \emptyset, X_0, \Theta_0)$.
Attach ν_0 to the initial NBA state $q_{B,0}$ and insert $q_{B,0}$ into the NBA-state expansion stack $\mathcal{S}_B$.
$\Pi_{\text{first}} \leftarrow \emptyset$.
while $\mathcal{S}_B \neq \emptyset$ **do**
  Pop the next NBA state q_B from $\mathcal{S}_B$.
  Initialize the current pending set $\mathcal{P}_B \leftarrow \emptyset$.
  Generate candidate NBA transitions $e_B : q_B \xrightarrow{\sigma} q'_B$ from the outgoing GBA transitions.
  for all candidate transition $e_B : q_B \xrightarrow{\sigma} q'_B$ **do**
    for all active plan nodes ν associated with q_B, in nondecreasing current plan cost **do**
      Compute the ordering score $S(\nu, e_B)$.
      $\nu' \leftarrow \text{ExtendPlanGraph}(G_P, \nu, e_B, \mathcal{R})$.
      if $\nu'.\text{Prog} \neq \text{dead}$ **then**
        Use $S(\nu, e_B)$ to update the retained score of q'_B in $\mathcal{P}_B$.
      end if
      if $\nu'.\text{Prog} = \text{acc}$ and $\Pi_{\text{first}} = \emptyset$ **then**
        Extract Π_{first} through the representative-predecessor chain.
      end if
    end for
    if q'_B is new and has at least one retained plan node **then**
      Add q'_B to $\mathcal{P}_B$.
    end if
  end for
  Order the states in $\mathcal{P}_B$ by their retained scores, using generation order to break ties.
  Insert the ordered states into $\mathcal{S}_B$.
  Mark q_B as expanded.
end while
return $(G_P, \Pi_{\text{first}})$.

be its required robot type, required number of robots, and target region. For each eligible robot $r_k \in \mathcal{R}_{\tau_a}$, the predicted arrival time is

$$\hat{t}(r_k, a \mid \nu_i) = \theta_i(r_k) + \frac{d_k(x_i(r_k), l_a)}{v(r_k)}, \tag{5}$$

where $d_k(x_i(r_k), l_a)$ is the shortest feasible travel distance for robot r_k from $x_i(r_k)$ to l_a. The n_a robots with the smallest feasible arrival times are selected, while enforcing that a robot is not selected twice within the same synchronized subtask. The selected robots are synchronized at the maximum of their arrival times, and their positions and completion times are updated in the successor node. If any required action proposition lacks enough eligible robots, the candidate successor is infeasible and is discarded.

When the expansion reaches a completed prefix-suffix structure, the temporary dead child is not retained as an ordinary plan node. Instead, its parent is recorded as a feasible solution leaf. The corresponding executable plan is then extracted through the representative-predecessor chain. If a newly generated anchor reaches a Buchi state whose outgoing structure has already been expanded, the Buchi state itself is not expanded again; instead, the new anchor is connected back to compatible existing successors at that state whenever possible. This late-anchor recovery allows additional execution contexts to reuse already generated successor structure without duplicating the whole suffix graph.

Algorithm 3 EXTENDPLANGRAPH: plan-graph extension for an NBA transition

Require: Plan graph G_P, parent node ν_i, NBA transition $e_B : q_{B,i} \xrightarrow{\sigma} q'_B$, and robot team $\mathcal{R}$.
Ensure: Successor plan node ν'.
Obtain the induced subtask $\omega = (\sigma, \sigma_i^s)$.
Compute the successor progress flag Prog' and the transition-label signature $\text{Sig}(\omega)$.
$\hat{\nu} \leftarrow$ compatible anchor at $(q'_B, \text{Prog}', \text{Sig}(\omega))$, if one exists.
$(\hat{\mathcal{C}}, \hat{X}, \hat{\Theta}, \hat{J}) \leftarrow \text{GREEDYASSIGN}(\omega, \nu_i, \mathcal{R})$.
if GREEDYASSIGN is infeasible **then**
  Set $\nu'.\text{Prog} \leftarrow \text{dead}$.
  return ν'.
end if
if $\hat{\nu}$ exists **then**
  Add the structural edge $(\nu_i, \hat{\nu})$ to E_P.
  if $\hat{J}$ improves the representative payload of $\hat{\nu}$ **then**
    Update $\text{par}(\hat{\nu})$, $\hat{\mathcal{C}}$, $\hat{X}$, and $\hat{\Theta}$; propagate the improvement to descendants.
  end if
  $\nu' \leftarrow \hat{\nu}$.
else
  Create $\nu' = (q'_B, \text{Prog}', \omega, \hat{\mathcal{C}}, \hat{X}, \hat{\Theta})$.
  Set $\text{par}(\nu') \leftarrow \nu_i$, insert ν' into V_P, and add (ν_i, ν') to E_P.
end if
if ν' completes an accepting prefix–suffix structure **then**
  Set $\nu'.\text{Prog} \leftarrow \text{acc}$.
  Record ν' as a feasible solution leaf.
end if
return ν'.

1) Child Node Generation: The plan graph is grown by extending a plan node ν_i along the outgoing NBA transitions of its associated state $q_{B,i}$. For each transition $e_B : q_{B,i} \xrightarrow{\sigma} q'_B$, the induced subtask is determined by the transition condition σ together with the self-loop condition σ_i^{s} at $q_{B,i}$, i.e., $\omega_j = (\sigma, \sigma_i^{\text{s}})$. The corresponding child handling then produces a successor associated with q'_B by extending the partial plan rooted at ν_i.

The successor state is evaluated through a greedy synchronized task-assignment procedure. Let $\mathcal{A}(\omega_j) \subseteq A$ denote the set of action propositions required by ω_j, and let $c(a_\ell) = (\tau_\ell, n_\ell, l_\ell)$ denote the execution requirement of $a_\ell \in \mathcal{A}(\omega_j)$, where τ_ℓ, n_ℓ, and l_ℓ are the required robot type, required number of robots, and target region, respectively. For each

candidate robot $r_k \in \mathcal{R}_{\tau_\ell}$, the predicted arrival time to l_ℓ is estimated by

$$\hat{\theta}_\ell(r_k) = \theta_i(r_k) + \frac{d_k(x_i(r_k), l_\ell)}{v(r_k)},$$

where $x_i(r_k)$ and $\theta_i(r_k)$ are the predicted position and predicted completion time of robot r_k at ν_i. The n_ℓ robots with the smallest predicted arrival times are selected, synchronized at the maximum of their predicted arrival times, and then used to update the predicted positions and completion times in the successor node. If fewer than n_ℓ eligible robots are available, the candidate is declared infeasible and discarded. The planning stage of the successor is updated according to the prefix–suffix progress of the current path: the expansion remains in the prefix stage until an accepting NBA state is reached, switches to the suffix stage when such a state is crossed from the prefix stage, and, once an accepting prefix–suffix execution is completed, immediately yields a feasible incumbent online. In this sense, child generation is guided by executable task assignments and their realized costs, while the detailed priority policy among generated successors is deferred to the traversal rules described next.

2) Residual-Obligation Scoring of Büchi State: To bias on-the-fly Büchi expansion toward successors with lower residual obligation pressure, we assign a lightweight heuristic score to each parent-transition pair (p, τ), where p is the current plan node and τ is a candidate Büchi transition. The score is computed from the remaining must-eventually propositions together with the current execution state carried by p.

we recursively extract a must-eventually set from the abstract syntax tree (AST) of the task formula. For the global LTL formula φ, we define a recursive mapping $\mathsf{ME}(\cdot)$ on the subformulas in the AST of φ. Here, ψ denotes an arbitrary subformula of φ. For such a subformula ψ, $\mathsf{ME}(\psi) \subseteq A$ denotes the set of positive action propositions that are extracted as unavoidable when ψ is recursively evaluated. The must-eventually set of the original task formula is then given by $M = \mathsf{ME}(\varphi)$.The recursive rules for computing $\mathsf{ME}(\psi)$ are defined as follows.

1) The constants $\top$ and $\bot$ do not introduce any positive action proposition that must be completed. That is, if $\psi = \top$ or $\psi = \bot$, then $\mathsf{ME}(\psi) = \emptyset$.
2) Only positive action propositions are included in the lower-bound estimate. A negated formula imposes a prohibition rather than a positive action proposition that must be completed, i.e. If $\psi = \neg\psi_1$, then $\mathsf{ME}(\psi) = \emptyset$ and if $\psi = a \in \mathcal{A}$, then $\mathsf{ME}(\psi) = a$
3) If $\psi = \psi_1 \wedge \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1) \cup \mathsf{ME}(\psi_2)$. Since both subformulas must be satisfied, the action propositions are collected from both sides.
4) If $\psi = \psi_1 \vee \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1) \cap \mathsf{ME}(\psi_2)$. Since either branch may be selected, only action propositions that are unavoidable in both branches are kept.
5) If $\psi = \bigcirc\psi_1$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1)$. The next operator shifts the satisfaction time but does not change the unavoidable positive action propositions.
6) If $\psi = \Box\psi_1$ or $\psi = \Diamond\psi_1$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1)$. The always and eventually operators preserve the positive action propositions appearing in the operand.

Based on the above recursive rules, the must-eventually set of the global task formula is obtained as M. The set M should not be interpreted as the set of action propositions that must be satisfied immediately at the current plan node. Instead, it provides a formula-level summary of unavoidable positive action propositions. This summary is used to estimate the minimum remaining effort when the future subtasks have not been fully generated during the incremental NBA construction.

Consider a parent plan node ν_i attached to q_B and a candidate transition $\tau : q_B \xrightarrow{\sigma} q'_B$. Let $A^+(\nu_i, \tau)$ be the progress summary obtained by combining the positive propositions already satisfied along the representative context with the positive propositions asserted by τ. The residual obligation set is

$$\mathcal{R}(\nu_i, \tau) = \mathcal{M} \setminus A^+(\nu_i, \tau). \tag{6}$$

For each $a \in \mathcal{R}(\nu_i, \tau)$ with $c(a) = (\tau_a, n_a, l_a)$, we compute the arrival-time estimates of all robots in R_{τ_a} from the current payload of ν_i and take the n_a-th smallest value, denoted by $h_a(\nu_i)$. The residual pressure of (ν_i, τ) is

$$H(\nu_i, \tau) = \begin{cases} \max_{a \in \mathcal{R}(\nu_i, \tau)} h_a(\nu_i), & \mathcal{R}(\nu_i, \tau) \neq \emptyset, \\ J_i, & \mathcal{R}(\nu_i, \tau) = \emptyset. \end{cases} \tag{7}$$

The candidate score is

$$S(\nu_i, \tau) = \max\{J_i, H(\nu_i, \tau)\}. \tag{8}$$

If the same successor Buchi state is discovered from multiple active anchors in the current pending batch, the smallest candidate score is retained for ordering that successor in the batch.

C. Traversal Rules

The shared plan graph is generated according to three rules. First, for a fixed Buchi state, active plan-node anchors are expanded in nondecreasing realized cost. This favors cheaper execution contexts when several partial plans reach the same Buchi state, but it remains a local rule for the current state.

Second, successor nodes are created through compatible-anchor reuse whenever possible. A successor anchor is compatible only when it matches the destination Buchi state, the prefix-suffix progress flag, and the transition-label signature. Reusing an anchor adds a structural graph edge. The representative predecessor and runtime payload of the shared node are changed only when the new parent yields a strictly lower realized cost, in which case the improvement is propagated to downstream successors.

Third, newly discovered successor Buchi states are collected in the current pending batch. After the current Buchi state has been processed, these successors are inserted into the later expansion stack according to their retained residual-obligation scores, with the original discovery order used as a tie breaker. Therefore, the score changes the local discovery order of newly generated Buchi states, rather than defining a global priority queue.

D. Final Post-Construction Pruning

After the shared plan graph has been constructed, a graph-level simplification pass is applied from the root. This pass does not minimize the Buchi automaton and does not claim to produce a globally minimal plan graph. Its purpose is to remove graph portions that cannot contribute to an accepting plan and to reduce local redundancy before branch-and-bound optimization. The simplification first identifies the live corridor. Starting from the root, it collects the reachable plan nodes and then propagates liveness backward from reachable accepting nodes or recorded solution leaves. A node is live if it is root-reachable and lies on at least one active path that can reach an accepting continuation. Edges to non-live nodes are removed. Nodes associated with hard-dead automaton states or internal cut marks are also removed when they do not support any live accepting corridor. Since the graph is shared, removing an edge does not immediately delete the child node; the child is retired only when it has no remaining structural parent.

The pass then performs local sibling-dominance simplification. For children of the same parent with the same local signature, a child edge can be removed if another sibling covers its successor structure with no larger realized cost. This operation is local to the shared plan graph and preserves the accepting corridors retained by the liveness test.

E. Plan Extraction

Whenever an accepting prefix-suffix structure is reached during construction, the corresponding solution leaf is recorded. Because the plan graph is shared, a node may have multiple structural parents. The executable plan, however, is extracted through the representative-predecessor chain. Starting from a recorded solution leaf ν_f, the algorithm repeatedly follows $\mathrm{par}(\nu)$ until the root node is reached, and then reverses the obtained sequence to form a root-to-leaf plan.

For each node on this representative path, the stored assignment map, predicted robot positions, and completion times define the executable subtask transition. Thus, the extracted plan is consistent with the runtime payload maintained in the shared graph. The structural parent set is used for graph sharing and simplification, but it is not enumerated during plan extraction.

VII. BRANCH-AND-BOUND OPTIMIZATION

After the plan graph is constructed and a feasible solution is available, we further refine the solution within the generated plan graph using a warm-started branch-and-bound (BnB) method. The objective is to minimize the task completion time, measured by the makespan of all robots, i.e. $\max_{r_k \in \mathcal{R}} ET_k$, where ET_k is the completion time of robot r_k. The best solution $J^\star$ is initialized by the feasible solution obtained from the plan graph, and BnB then systematically explores the assignment space to improve. If the search frontier is fully exhausted, the best solution is certified to be optimal within the generated plan-graph assignment space. Otherwise, if the search is terminated early due to runtime or memory limits, the algorithm returns the best feasible solution found so far.

Algorithm 4 Warm-Started BnB on the Generated Plan Graph

Require: Plan graph $G_P = (V_P, E_P)$, incumbent plan $\Pi^\star$, incumbent cost $J^\star$.
Ensure: Improved plan $\Pi^\star$ and cost $J^\star$.

```
Initialize the root BnB node μ_0 = (ν_0, X_0, Θ_0, C_∅).
Insert μ_0 into a priority queue Q with key LB(μ_0).
while Q ≠ ∅ and the stopping limit is not reached do
    Pop μ_i = (ν_i, X_i, Θ_i, C_i) with the smallest lower bound from Q.
    if LB(μ_i) ≥ J* then
        continue
    end if
    if ν_i is an accepting terminal plan node then
        if J_cur(μ_i) < J* then
            Update (Π*, J*) using the plan represented by μ_i.
        end if
        continue
    end if
    for all ν_j such that (ν_i, ν_j) ∈ E_P do
        for all feasible assignments C_j for the subtask associated with ν_j do
            Propagate robot positions and completion times to obtain
                        μ_j = (ν_j, X_j, Θ_j, C_i ∪ C_j).
            if LB(μ_j) < J* then
                Insert μ_j into Q with key LB(μ_j).
                Use a greedy rollout from μ_j to update (Π*, J*) if a better feasible completion is found.
            end if
        end for
    end for
end while
return (Π*, J*).
```

A. Node Expansion

Each BnB search node is represented as below.

$$\mu_i = (\nu_i, X_i, \Theta_i, \mathcal{C}_i), \tag{9}$$

where ν_i is the current plan node in the generated plan graph, vector $X_i = (x_i(r_1), x_i(r_2), \ldots, x_i(r_{n_r}))$ denotes the predicted robot positions. The vector $\Theta_i = (\theta_i(r_1), \theta_i(r_2), \ldots, \theta_i(r_{n_r}))$ denotes the predicted completion times of robots, and $\mathcal{C}_i$ is the assignment plan.

The expansion of a BnB node follows the connectivity of the plan graph. For a node $\mu_i = (\nu_i, X_i, \Theta_i, \mathcal{C}_i)$, only successor plan nodes ν_j satisfying $(\nu_i, \nu_j) \in E_P$ are considered, where E_P is the edge set of the plan graph. The child node μ_j of μ_i is created by assigning the subtask associated with ν_j to all feasible robot combinations. For an assignment plan $\mathcal{C}_j$ of μ_j, if robot r_k is assigned to an action proposition $a_\ell \in \mathcal{A}(\omega_j)$, its predicted position $x_j(r_k) \in X_j$ and completion time $\theta_j(r_k) \in \Theta_j$ are computed in the same manner as in Sec. VI-B1. Robots not involved in the subtask inherit their parent states from X_i and Θ_i.

B. Lower Bound

For a BnB node μ_i, the lower bound estimates the smallest possible completion time that can be achieved from μ_i after completing the remaining subtasks in the generated plan graph

Let $\mathcal{A}_{\mathrm{rem}}(\nu_i)$ denote the set of action propositions appearing in the remaining subtasks reachable from ν_i. If $\mathcal{A}_{\mathrm{rem}}(\nu_i) = \emptyset$, then

$$LB(\mu_i) = J_{\mathrm{cur}}(\mu_i). \tag{10}$$

Here, $LB(\mu_i)$ denotes the lower-bound estimate of the final makespan for all feasible continuations rooted at μ_i, and $J_{\mathrm{cur}}(\mu_i)$ denotes the current makespan at μ_i. For each action proposition $a \in \mathcal{A}_{\mathrm{rem}}(\nu_i)$, let $c(a) = (\tau_a, n_a, l_a)$ be its execution requirement, where τ_a is the required robot type, n_a is the required number of robots, and l_a is the target region. For each type-compatible robot $r_k \in \mathcal{R}_{\tau_a}$, we compute the relaxed arrival-time estimate

$$\widehat{\theta}_i(r_k, a) = \theta_i(r_k) + \frac{d_k(x_i(r_k), l_a)}{v(r_k)}, \tag{11}$$

where $x_i(r_k) \in X_i$ is the predicted position of robot r_k, $v(r_k)$ is its constant travel speed, and $d_k(x_i(r_k), l_a)$ denotes its shortest feasible travel distance from $x_i(r_k)$ to l_a. Let $\ell_i(a)$ be the n_a-th smallest value among $\{\widehat{\theta}_i(r_k, a) \mid r_k \in \mathcal{R}_{\tau_a}\}$. If fewer than n_a type-compatible robots can reach l_a, then $\ell_i(a) = +\infty$.

The lower bound of μ_i is then defined as

$$LB(\mu_i) = \max\{J_{\mathrm{cur}}(\mu_i), \max_{a \in \mathcal{A}_{\mathrm{rem}}(\nu_i)} \ell_i(a)\}. \tag{12}$$

This bound is obtained by relaxing the remaining assignment problem: it ignores the ordering among future subtasks and the reuse constraints of robots across multiple future subtasks. Therefore, any complete feasible solution rooted at μ_i must have a final makespan no smaller than $LB(\mu_i)$. If $LB(\mu_i) \geq J^\star$, where $J^\star$ is the incumbent best cost, then μ_i cannot improve the incumbent solution and is pruned.

C. Upper Bound

After a child BnB node is generated, BnB attempts a greedy rollout from it to obtain a feasible complete solution and tighten the best upper bound. A plan node may have multiple successor plan nodes. Performing a complete rollout for every successor would introduce considerable computational overhead and would weaken the lightweight nature of the upper-bound estimation. Therefore, at each rollout step, only one promising successor is selected according to a fast residual-task estimate.

Specifically, for the current rollout node μ_i, the algorithm considers the successor plan nodes μ_j. For each candidate successor, the subtask associated with μ_j is greedily assigned to a feasible robot combination, resulting in a temporary child node μ_j. The candidate is then evaluated using the remaining action propositions reachable from μ_j. The successor with the smallest estimated completion pressure is selected for the rollout, i.e., $\nu_j^\star \in \arg\min_{(\nu_i,\nu_j)\in E_P} \widehat{J}(\mu_j)$, where $\widehat{J}(\mu_j)$ is computed in the same manner as the residual action-proposition estimate used in the lower-bound calculation. This selection rule favors the successor whose remaining subtasks are estimated to be completed with the smallest makespan, without enumerating all suffix continuations in the plan graph.

The rollout then proceeds from $\mu_j^\star$ and repeats the same successor-selection and greedy-assignment procedure until a terminal accepting plan node is reached or the rollout fails. A rollout is accepted only if it reaches a terminal accepting plan node and all synchronized assignments along the rollout are valid. When a valid rollout produces a makespan $J_{\mathrm{roll}} < J^\star$, the best solution is updated as $J^\star \leftarrow J_{\mathrm{roll}}$, together with the corresponding assignment plan and plan-node path. The rollout is used only to improve the incumbent upper bound and does not certify optimality.

D. Branching and Pruning

The search maintains an open list in a best-first manner. Each generated BnB node is ranked according to the lower-bound estimate, which estimates the smallest possible final makespan that can be achieved by completing the remaining subtasks from the current node. Specifically, for a generated node μ_j, let $\underline{J}(\mu_j)$ denote this estimated lower bound. The open list prioritizes the node with the smallest $\underline{J}(\mu_j)$, so that the most promising partial assignment plan is expanded first. If $\underline{J}(\mu_i) \geq J^\star$), the node cannot improve the best solution and is pruned. If the type or cardinality requirement of a remaining subtask cannot be satisfied, the node is removed as infeasible. Otherwise, it is inserted into the open list for further exploration.

VIII. Plan Adaptation under Agent Failures

During task execution, a robot may become unavailable because of an unexpected failure and can no longer complete its assigned subtasks. The unfinished part of the mission must therefore be replanned using the remaining robot team.

The unavailable robots are first removed from the robot team. The current execution progress and the states of the remaining robots then define a new root for replanning. The replanning search is initialized without a warm start because the previous assignment may involve the unavailable robot. Starting from the new root, the branch-and-bound procedure reassigns the unfinished subtasks over the retained plan graph while enforcing the original temporal and collaboration constraints. During the search, the upper-bound procedure uses a greedy rollout to construct a feasible completion. This allows the planner to obtain the first adapted solution quickly. The solution then serves as the incumbent for subsequent improvement. The final assignment replaces the unexecuted part of the original plan.

IX. Complexity Analysis

This section analyzes the computational complexity of the proposed framework by decomposing it into three stages: GBA construction with pruning, on-the-fly NBA and plan-graph construction, and the final BnB assignment search.

Let n_G be the number of GBA states after the inherited LTL2BA simplifications, and let d_G be the maximum GBA out-degree. The proposed ST pruning checks whether a direct

transition can be replaced by a two-hop transition while preserving the corresponding acceptance progress. Its additional worst-case cost is $O(n_G d_G^3)$. Let C_s candidate transition labels are generated at a GBA state s, and T_f denotes the cost of one feasibility check, the feasibility screening cost is $O(\sum_{s\in S_G} C_s T_f)$.

During NBA generation, the implementation constructs the plan graph on the fly. Each generated NBA transition induces a candidate plan node. If the induced task cannot be assigned to the current robot team, the corresponding branch is discarded. Let N_P denote the number of plan node candidates evaluated during this on-the-fly construction. For one plan node, the main computational complexity comes from node evaluation and greedy assignment. If a candidate node contains m atomic propositions requirements and the i-th AP is associated with robot type τ_i, the complexity of greedy assignment part is $O(m\log m + \sum_{i=1}^{m} \mathcal{R}(\tau_i)(C_{\text{dist}} + \log \mathcal{R}(\tau_i) + \mathcal{R}))$, where C_{dist} is the cost of one distance query and is $O(1)$ when the distance value is cached. Let A_s be the number of plan anchors currently stored at the target BState, and let B be the length of the AP bit-set. The complexity of node evaluation is $O\Big((A_s+1)B + \sum_{j=1}^{q} \mathcal{R}(\tau_j)(C_{\text{dist}} + \log \mathcal{R}(\tau_j))\Big)$.

The bnb stage searches over robot assignments on the generated plan graph. For a plan-graph node ν, suppose that its corresponding subtask contains m_ν action requirements, where the i-th requirement selects k_i robots from r_i compatible candidates. Let $K_\nu = \sum_i k_i$. The assignment branching factor at ν is bounded by $M_\nu \le \prod_{i=1}^{m_\nu} \binom{r_i}{k_i}$. The lower bound calculation first collects future required propositions from the current bnb node and its children, and then estimates the earliest arrival time of the required compatible robots. If d_ν is the number of child nodes, h_ν is the number of future required propositions collected by this lower-bound routine, and $B = \lceil |\mathcal{AP}|/w \rceil$ is the AP bit-set length in machine words, then the complexity of lower bound calculation is $O\Big((d_\nu+1)B + \sum_{\ell=1}^{h_\nu} \mathcal{R}(\tau_\ell)(C_{\text{dist}} + \log \mathcal{R}(\tau_\ell))\Big)$. Upper-bound updates follow one heuristic path in the plan graph rather than enumerating all assignments. If the rollout visits L_{roll} plan nodes, then the complexity is $O\Big(\sum_{t=1}^{L_{\text{roll}}} (d_t C_{\text{heur}}(t) + K_t(C_{\text{dist}}+1) + R)\Big)$, where $C_{\text{heur}}(t)$ is the cost of scoring one child during rollout. If g_t required regions are considered by the heuristic, a coarse bound is $C_{\text{heur}}(t) = O((d_t+1)B + R g_t C_{\text{dist}})$ which accounts for required-region collection and robot-region arrival-time checks.

Overall, the LTL formula complexity and the robot-team size affect different parts of the framework. A more complex formula usually increases the AP set and the automaton branching structure, which is reflected by n_G, d_G, B, N_P, and the plan-graph quantities d_ν, h_ν, and L_{roll}. These terms determine how many automaton transitions, plan-node candidates, and future AP requirements must be examined. In contrast, the number of robots mainly affects the assignment terms. During NBA/plan-graph construction, the greedy evaluation of one candidate grows roughly with $\sum_i \mathcal{R}(\tau_i)\log \mathcal{R}(\tau_i)$ when distance queries are cached. In the bnb stage, adding compatible robots increases r_i and therefore the branching factor $M_\nu = \prod_i \binom{r_i}{k_i}$. If each k_i is fixed, this factor grows polynomially with degree $K_\nu = \sum_i k_i$; if the number of collaborative requirements or interchangeable robots grows, the product becomes combinatorial. Therefore, the dominant cost is jointly controlled by N_P from the formula/automaton side and by M_ν, N_A, and H from the robot-assignment side. The proposed pruning and incumbent updates reduce these effective quantities in practice, but they do not remove the worst-case combinatorial dependence of synchronized multi-robot assignment.

X. THEORETICAL ANALYSIS

Lemma 3. *The offline NBA optimization inherited from LTL2BA preserves feasible solutions.*

Proof. The offline optimization removes only unreachable states, states that cannot participate in an accepting run, or transitions that are redundant under the standard language-preserving LTL2BA simplifications. Such elements cannot be part of a feasible accepting run. Therefore, any feasible solution before the offline optimization has an equivalent accepting run after the optimization. □

Lemma 4. *The on-the-fly plan-graph construction algorithm is complete, i.e., if a feasible solution exists, the initial algorithm is guaranteed to find it.*

Proof. Let Π_φ^{fin} be a feasible finite execution in the sense of the problem definition, and let $q_{B,0}q_{B,1}\cdots q_{B,n}$ be the corresponding finite segment of the accepting NBA run ρ_φ, where $q_{B,i} = (q_g^i, \ell_i)$. For each NBA transition $e_B^i : q_{B,i} \xrightarrow{\sigma_i} q_{B,i+1}$, the on-the-fly construction generates the induced subtask T_{i+1} and either creates a successor plan node ν_{i+1} or reconnects to an equivalent semantic anchor carrying the same $q_{B,i+1}$, stage, and transition signature.

The construction discards a successor only when its synchronized subtask is infeasible for the current robot team. Since Π_φ^{fin} is feasible, every subtask along this execution admits a valid synchronized assignment and is not removed by this feasibility test. Therefore, the full accepting sequence remains represented in the plan graph, and a feasible initial plan can be obtained. □

Lemma 5. *Every solution returned by the branch-and-bound search satisfies the constraints of the plan graph.*

Proof. The branch-and-bound search is initialized from nodes in the generated plan graph and expands states only along plan-graph edges. For each synchronized task, the assignment enumerator selects the required number of distinct robots from the compatible robot set and updates their positions, execution times, and task records accordingly. The rollout procedure also follows plan-graph edges and replays stored feasible assignments. Thus every returned solution corresponds to a valid path in the plan graph and satisfies both the automaton constraints and the robot-assignment constraints. □

Theorem 1. *Given sufficient time, if a feasible execution exists, the proposed algorithm obtains a feasible solution and eventually returns one that minimizes the completion time.*

Proof. Let $\mathcal{F}$ be the set of feasible finite executions that satisfy φ and all robot-assignment constraints, and let $T^\star = \min_{\Pi\in\mathcal{F}} T(\Pi)$. First, suppose $\mathcal{F} \neq \emptyset$. By Lemmas **??**–4, automaton pruning and on-the-fly plan-graph construction preserve at least one feasible accepting execution. The branch-and-bound stage expands only plan-graph edges and enumerates compatible robot assignments. Hence, with sufficient time, it reaches a terminal accepting plan or obtains one through rollout, and records a finite incumbent cost $J^\star$. Therefore, whenever a feasible solution exists, the algorithm can obtain one.

It remains to show optimality with respect to completion time. Assume that the plan graph representing feasible prefix–suffix executions is finite and that the lower bound $LB(\nu)$ used by branch-and-bound is admissible, i.e., it never exceeds the minimum feasible completion time of any plan extending node ν. Let $\Pi^\star \in \mathcal{F}$ be an execution with completion time $T^\star$. For every node ν on the branch leading to $\Pi^\star$, admissibility gives $LB(\nu) \leq T^\star$. If the current incumbent satisfies $J^\star > T^\star$, then such a node cannot be pruned by the rule $LB(\nu) \geq J^\star$. If $J^\star = T^\star$, an execution with minimum completion time has already been found.

Thus, before the optimum is found, the branch leading to $\Pi^\star$ remains in the search frontier. Given sufficient time, branch-and-bound eventually explores this branch and updates the incumbent to $J^\star = T^\star$. Once the smallest frontier lower bound is no smaller than $J^\star$, no unexplored branch can yield a completion time below $T^\star$. Consequently, the returned solution minimizes $T(\Pi^{\text{fin}}_\varphi)$. □

XI. NUMERICAL EXPERIMENTS

This section evaluates the proposed framework through six case studies: (i) we evaluate the performance of the warm-started branch-and-bound (BnB) optimizer to examine whether the initial feasible solution can be progressively improved; ii) we study the effect of incorporating planning information during automaton generation and show that planning-aware pruning and on-the-fly construction help obtain the first feasible solution more quickly; iii) we compare the first feasible solution returned by the proposed method with the optimized solution to examine the quality of the initial plan; iv) we vary the number of robots to evaluate the scalability of the proposed framework; v) we compare our method with poset- and MILP-based methods to evaluate its performance across different LTL specifications and robot-team sizes.; vi) we test the framework in robot-unavailability scenarios and demonstrate its ability to repair the remaining mission online during execution.

All algorithms were implemented in Cython 0.29.24 and executed on a workstation with 3.40-GHz Gen Intel(R) Core(TM) i7-11390H and 16 GB RAM. All experiments are conducted in a $(40\text{m} \times 40\text{m})$ workspace consisting of eight areas of interest, with obstacles randomly generated in the workspace, as shown in Fig. 1. Unless otherwise stated, each reported runtime is measured from the submission of the LTL formula to the generation of the corresponding plan. We report the time to the first feasible solution, denoted by T_{first}, the time when the best solution is obtained, denoted by T_{best}, and the total search time, denoted by T_{search}.

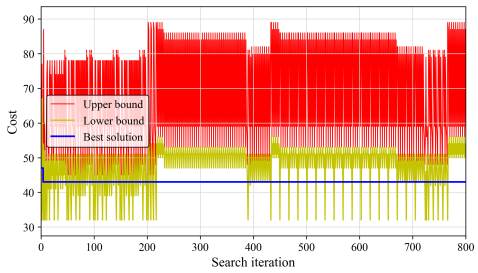

Fig. 2. The evolution of the upper bound, lower bound, and optimal value.

A. Warm-Started Branch-and-Bound Optimization

This case study examines whether the warm-started BnB optimizer can effectively improve the solution quality after the first feasible plan is obtained from the on-the-fly plan graph. The task formula is $\varphi = \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond(a_6 \wedge \Diamond(a_4 \vee a_5))) \wedge \Diamond a_7 \wedge \Diamond a_9 \wedge \Diamond a_{10}$. Specifically, $c(a_1) = (1, 3, l_1), c(a_2) = (2, 4, l_2), c(a_3) = (2, 1, l_3), c(a_4) = (1, \lfloor n/2 \rfloor, l_4), c(a_5) = (5, \lceil 2n/3 \rceil, l_5), c(a_6) = (3, \max\{2, \lceil 0.75n \rceil\}, l_6)$, $c(a_7) = (1, 3, l_7), c(a_8) = (1, 2, l_8), c(a_9) = (3, 3, l_8), c(a_{10}) = (4, 1, l_8)$. Here, $n = 5$ and the robot team consists of five types of robots, with five robots of each type. The initial positions of robots are randomly generated in the workspace.

As shown in Fig. 2, the initial plan graph returns a feasible solution with $J_{\text{first}} = 47.21$. The first pruning event occurs at 0.2910 s. This is mainly because that the warm start provides an initial solution at the beginning of the search, which enables a large number of assignment nodes to be pruned early. Overall, the BnB search prunes 122,362 out of 138,145 visited assignment nodes, corresponding to a pruning ratio of 88.57%. The best solution with cost $J_{\text{best}} = 43.32$ is obtained at 2.3679 s. This is because during the BnB search, the estimated lower bound is used as the priority criterion for selecting branching nodes. This best-first strategy biases the search toward partial assignments with more promising current and future costs, allowing the optimizer to find the best solution after exploring only a limited portion of the assignment space.

B. Effect of Planning-Aware Automaton Construction

This case study isolates the contribution of planning-aware automaton construction. We use the same robot team as Sec.XI-A and consider formulas of the form $\bigwedge_{i=1}^{m} \Diamond a_i$, where m is the number of atomic propositions. Specifically, for $(i = 1, \ldots, 8)$, $c(a_i) = (1, 3, l_i)$, and for $(i = 9, \ldots, 15)$, $c(a_i) = (2, 3, l_{i-8})$.] Thus, $(a_1, \ldots, a_8)$ require three type-1 robots to visit regions $(l_1, \ldots, l_8)$, respectively, while $(a_9, \ldots, a_{15})$ require three type-2 robots to visit regions $(l_1, \ldots, l_7)$, respectively. The proposed method is compared with two variants. The initial positions of robots are randomly generated in the workspace. The first variant retains the proposed GBA-level pruning but disables on-the-fly planning, so that planning is performed only after the NBA has been fully constructed. The

TABLE I
EFFECT OF ON-THE-FLY PLANNING ON FIRST-SOLUTION TIME

m	Proposed	Variant I
12	10.4065	12.7062
13	30.7849	53.2725
14	156.0395	197.4069
15	723.6905	1037.2168

second variant disables both GBA-level pruning and on-the-fly planning during automaton generation; it first constructs the complete NBA and then applies the same feasibility and decomposability pruning as a post-processing step.

Tables I and II demonstrate the benefit of integrating pruning and planning into the automaton-generation process. Table **??** isolates the effect of on-the-fly planning when GBA-level pruning is retained. The proposed method consistently obtains the first feasible solution faster than Variant I, which postpones planning until the NBA is fully constructed. This advantage becomes important for longer specifications, because the cost of complete NBA construction grows rapidly with the number of atomic propositions. Instead of waiting for the full NBA, the proposed method constructs the plan graph during NBA generation and can return a feasible plan as soon as an accepting prefix-suffix structure is discovered.

Table II further shows that early pruning at the GBA level avoids constructing many NBA transitions that would later be removed. For example, when (m=15), Variant II first generates an NBA with ((32768,737280)) states and transitions, whereas the proposed method directly generates the pruned NBA with ((32768,311296)). Although post-processing eventually reduces Variant II to the same NBA size, it has already paid the cost of constructing the larger unpruned NBA. Consequently, its first-solution time is much longer, increasing from 723.6905 s for the proposed method to 1235.5359 s. These results indicate that pruning after complete NBA construction can reduce the final automaton size, but it cannot recover the time wasted on generating unnecessary transitions. Therefore, the proposed framework benefits from both components: GBA-level pruning reduces the automaton construction burden, while on-the-fly planning enables the first feasible plan to be obtained before the complete NBA needs to be explored.

C. First-Solution Quality

This case study evaluates whether the first feasible solution generated by the on-the-fly plan graph is already of useful quality. Four representative LTL formulas are tested, and each case is executed 25 times. Specifically, we consider

$$\begin{aligned}
\varphi_1 &= \Diamond(a_2 \wedge \Diamond a_1) \wedge \Diamond(a_3 \wedge \Diamond(a_5 \wedge \neg p_5) \\
&\quad \wedge \Diamond a_4 \wedge \Diamond a_6 \wedge \Diamond a_7 \wedge \Diamond a_8, \\
\varphi_2 &= \Diamond a_2 \wedge \Diamond a_1 \wedge \Diamond a_3 \wedge \Diamond(a_3 \vee a_4) \\
&\quad \wedge \Diamond a_7 \wedge \Box\Diamond a_5, \\
\varphi_3 &= \Diamond a_9 \wedge \Diamond a_{10} \wedge \Diamond(a_3 \wedge a_4) \wedge \Diamond a_5 \\
&\quad \wedge \Box\neg p_1 \wedge \Diamond(a_7 \wedge \Diamond a_8) \\
&\quad \wedge (\neg a_5 \,\mathcal{U}\, (a_3 \wedge a_4)), \\
\varphi_4 &= \Diamond a_9 \wedge \Diamond a_{10} \wedge \Diamond a_{11} \wedge (\neg a_1 \,\mathcal{U}\, a_2) \\
&\quad \wedge \Diamond a_2 \wedge \Diamond a_1 \wedge \Diamond a_5 \wedge \Diamond a_4 \\
&\quad \wedge (\Box a_4 \,\mathcal{U}\, a_5).
\end{aligned} \tag{13}$$

These specifications involve various temporal operators and are representative of complex tasks commonly encountered in large-scale factory scenarios. For example, φ_4 is adapted from the multi-robot pickup and delivery (MRPD) task [24]. Each action proposition a_i is associated with an execution requirement $c(a_i) = (\tau_i, n_i, l_i)$, where τ_i is the required robot type, n_i is the required number of robots, and l_i is the target region. Specifically, $c(a_1) = (2, 3, l_1), c(a_2) = (1, 2, l_8), c(a_3) = (1, 1, l_2), c(a_4) = (2, 3, l_4), c(a_5) = (1, 3, l_7), c(a_6) = (1, 2, l_6), c(a_7) = (1, 3, l_7), c(a_8) = (1, 3, l_8), c(a_9) = (1, 3, l_5), c(a_{10}) = (1, 3, l_3), c(a_{11}) = (1, 3, l_1)$.

As shown in Tab.III, in two out of four cases, the first solution coincides with the best solution found in the generated plan graph. In the remaining two cases, the first solution is only 1.83% and 4.47% worse than the corresponding plan-graph best solution, respectively. Meanwhile, GBA pruning removes 40.24%-81.48% of the transitions, and the final graph simplification removes 5.00%-54.93% of the plan-graph edges. These results support the residual obligation ordering and shared plan-graph design: the method tends to discover a high-quality executable branch early while keeping the generated graph compact for subsequent optimization.

D. Scalability

This case study evaluates scalability with respect to the number of robots. The task formula is $\varphi = \Diamond(a_1 \vee a_2) \wedge \Diamond(a_3 \wedge a_4) \wedge \Diamond a_5 \wedge (\neg a_5 \,\mathcal{U}\, (a_3 \wedge a_4))$. All action propositions in this case require type-1 robots. The execution requirement of each action is defined as $c(a_i) = (\tau_i, n_i, l_i)$, where τ_i is the required robot type, n_i is the required number of robots, and l_i is the target region. Specifically, $c(a_1) = (1, n, l_2), c(a_2) = (1, n, l_5), c(a_3) = (1, \lfloor n/3 \rfloor, l_4), c(a_4) = (1, 1, l_4), c(a_5) = (1, 1, l_5)$. Here, the robot team consists only of type-1 robots, and n denotes the total number of available type-1 robots.

Table IV reports the cost and runtime of the first feasible solution, the best solution found by BnB, and the total search time. The proposed method obtains the first feasible solution quickly for all tested robot-team sizes. Even when (n=100), the first feasible solution is generated in 1.7447 s. This shows that the on-the-fly plan-graph construction remains effective for quickly producing executable plans as the robot team grows.

On the other hand, the total search time increases rapidly with the number of robots. For $n = 5$, the search is completed in 12.78 s, while for $n = 20$, it increases to 1426.57 s.

TABLE II
EFFECT OF EARLY PRUNING ON NBA SIZE AND FIRST-SOLUTION TIME

m	Proposed NBA	Proposed T_{first}	V-II NBA Before	V-II NBA After	V-II T_{first}
10	$(1024, 6144)$	1.0016	$(1024, 10240)$	$(1024, 6144)$	1.2243
12	$(4096, 28672)$	10.4065	$(4096, 61440)$	$(4096, 28672)$	13.5997
13	$(8192, 61440)$	30.7849	$(8192, 143360)$	$(8192, 61440)$	68.6376
14	$(16384, 131072)$	156.0395	$(16384, 327680)$	$(16384, 131072)$	239.8623
15	$(32768, 311296)$	723.6905	$(32768, 737280)$	$(32768, 311296)$	1235.5359

TABLE III
QUALITY OF THE FIRST FEASIBLE SOLUTION

Case	First Solution		Plan-Graph Best		Global Optimum		GBA Before/After	Plan Graph Before/After
	Cost	Time (s)	Cost	Time (s)	Cost	Time (s)		
1	125.23	0.1702	125.23	0.1702	102.67	1.3557	$(144, 2916)/(144, 640)$	$(712, 2232)/(622, 1847)$
2	74.06	0.0619	74.06	0.0619	67.73	0.1522	$(16, 82)/(16, 49)$	$(37, 60)/(35, 57)$
3	173.36	0.0807	170.24	0.1071	170.24	0.1071	$(36, 324)/(20, 60)$	$(43, 71)/(22, 32)$
4	212.98	0.4864	203.87	0.5127	176.93	1.0008	$(97, 1044)/(81, 291)$	$(212, 466)/(131, 299)$

TABLE IV
EFFECT OF THE NUMBER OF ROBOTS

n	J_{first}	T_{first} (s)	J_{best}	T_{best} (s)	T_{search} (s)
5	47	0.1427	43	0.6612	12.779979
10	46	0.1977	41	1.1977	263.79
20	28	0.4176	27	1.9690	1426.57
50	22	0.8604	22	0.8604	∞
100	23	1.74468	23	1.74468	∞

When $n = 50$ and $n = 100$, the search is not fully exhausted within the allowed time. This trend is consistent with the complexity analysis. During plan-graph construction, the greedy evaluation of a candidate node grows mainly with the number of compatible robots, so the first-solution time increases moderately. In contrast, the BnB stage must explore robot assignments whose branching factor is bounded by products of combinatorial terms such as $\prod_i \binom{r_i}{k_i}$. As n increases, the number of compatible robots and the required cardinalitie also increase, causing the assignment space to grow combinatorially.

These results demonstrate the anytime nature of the proposed framework. The method can return a feasible solution early even for large robot teams, while additional computation can be used to improve or certify the solution when the assignment space is not too large. For larger teams, complete enumeration becomes difficult, but the planner still provides an executable solution within a short time.

E. Comparison With Baselines

This case study compares the proposed method with a poset-based [3]method and an MILP-based method [7] on the same LTL formulas and robot-team configurations. The comparison focuses on both the first-solution latency and the final cost. We consider four LTL specifications:

$$\begin{aligned}
\varphi_1 &= \Diamond a_1 \wedge \Diamond(a_2 \wedge \Diamond(a_3 \wedge \Diamond a_4)) \wedge \Box\Diamond a_7,\\
\varphi_2 &= \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond(a_6 \wedge \Diamond(a_4 \vee a_5)))\\
&\quad \wedge \Diamond a_7 \wedge \Diamond a_8 \wedge \Diamond(a_9 \wedge \Diamond a_{10}),\\
\varphi_3 &= \Diamond a_1 \wedge \Diamond a_2 \wedge \Diamond(a_3 \wedge \Diamond a_4) \wedge \Diamond(a_5 \wedge \neg p_7)\\
&\quad \wedge \Diamond a_6 \wedge \Diamond a_9 \wedge (\Box\neg a_9 \mathcal{U} a_6),\\
\varphi_4 &= \Diamond(a_1 \wedge \Diamond a_2) \wedge \Diamond a_3 \wedge \Diamond a_6 \wedge \Diamond(a_4 \vee a_5)\\
&\quad \wedge \Diamond a_8 \wedge \Diamond a_7 \wedge \Diamond a_9 \wedge \Diamond a_{10} \wedge (\neg a_9 \mathcal{U} a_2).
\end{aligned}$$

Specifically, $c(a_1) = (1, 2, l_1), c(a_2) = (2, n, l_2), c(a_3) = (2, 1, l_3), c(a_4) = (1, \lfloor n/2 \rfloor, l_4), c(a_5) = (5, \lceil 2n/3 \rceil, l_5), c(a_6) = (3, \max 2, \lceil 0.75n \rceil, l_6), c(a_7) = (1, n, l_7), c(a_8) = (1, 2, l_8), c(a_9) = (3, 1, l_8), c(a_{10}) = (4, 1, l_8)$. These specifications are used to evaluate the runtime and solution quality of different methods under increasing task complexity. The robot team consists of five robot types. We vary the number of robots of each type n to test different team sizes. For example, let $n = 10$, 5×10 denotes a team with five types of robots and ten robots of each type, i.e., 50 robots in total.

As shown in Tab.V, the results show that the proposed method consistently obtains the first feasible solution much faster than the two baselines. This advantage becomes more significant for more complex specifications. For example, under φ_2 with a 5×20 robot team, the proposed method returns the first feasible solution in 1.1497 s, while the poset-based and MILP-based methods require 90.137 s and 67.0037 s, respectively. All three methods eventually reach the same best cost 48.931, but the proposed method reaches its best solution in 3.9071 s, much earlier than the poset-based method and the MILP-based method. Similarly, for the more complex specification φ_4, the proposed method obtains the first solution within 1.7783 s for the 5×20 team, whereas the poset-based and MILP-based methods require 270.42 s and 483.96985 s, respectively.

These results indicate that the proposed framework is particularly effective when an executable plan is needed early. The poset-based method needs to extract and reason over partial-order structures before task allocation, and the MILP-

TABLE V
COMPARISON WITH POSET- AND MILP-BASED BASELINES

Formula	Team Size	Method	T_{first} (s)	T_{best} (s)	T_{search} (s)	J_{first}	J_{best}
φ_1	5×5	Proposed	0.2104	0.3041	0.9299	57.21	51.48
		Poset	3.3264	7.0590	11.059	58.96	51.48
		MILP	2.1735	4.2144	59.1163	54.38	32.85
φ_2	5×5	Proposed	0.6987	3.9554	927.48	57.773	50.98
		Poset	89.203	91.287	940.062	62.45	50.98
		MILP	51.4076	60.8376	632.27	66.326	50.98
φ_2	5×10	Proposed	0.7216	4.9678	1378.92	50.168	47.89
		Poset	89.43	92.984	1305.835	48.78	47.89
		MILP	59.7567	62.2364	1198.765	56.34	47.89
φ_2	5×20	Proposed	1.1497	3.9071	8590.59	49.987	48.931
		Poset	90.137	97.8213	8697.43	49.076	48.931
		MILP	67.0037	74.0153	2507.36	65.32	48.931
φ_3	5×5	Proposed	0.3890	4.2470	157.28	38.247	33.446
		Poset	17.921	24.587	174.114	53.32	33.446
		MILP	6.5013	11.8756	336.4283	46.82	33.446
φ_3	5×10	Proposed	0.3089	8.7278	203.29	35.987	32.58
		Poset	18.385	18.985	247.815	33.48	32.58
		MILP	6.8304	13.9769	742.93	45.32	32.58
φ_3	5×20	Proposed	0.4203	11.8629	421.68	44.026	33.578
		Poset	17.124	24.8491	463.769	45.7	33.578
		MILP	13.9042	18.7462	1859.54	72.89	33.578
φ_4	5×5	Proposed	1.3311	4.5280	9972.67	69.46	57.42
		Poset	267.98	268.25	9903.93	67.64	57.42
		MILP	349.1159	356.4569	1093.42	71.326	57.42
φ_4	5×10	Proposed	1.7716	6.8869	∞	47.93	35.71
		Poset	268.98	269.54	∞	48.96	35.71
		MILP	371.2282	425.7460	3467.98	84.56	35.71
φ_4	5×20	Proposed	1.7783	7.0542	∞	42.89	33.89
		Poset	270.42	271.85	∞	39.67	33.89
		MILP	483.9699	508.9515	9075.42	44.54	33.89

based method must solve a large combinatorial optimization problem before returning a feasible assignment. In contrast, the proposed method constructs the NBA and the plan graph on the fly, so a feasible prefix-suffix plan can be returned as soon as a valid accepting structure is discovered. This explains the large reduction in T_{first}

F. Online Repair Under Robot Unavailability

This case study evaluates online repair when the available robot set changes during execution. The original LTL mission remains unchanged, but assignments involving unavailable robots become invalid and the remaining mission must be repaired from the current execution state. We use the same task specification, AP setting, workspace, and robot-team configuration as in Sec.XI-A. Two robot-removal events are triggered: robot 0 is removed at $t = 15.0$ s and robot 20 is removed at $t = 30.0$ s.

The proposed workflow generates repaired first solutions within 45.738 ms and 1.652 ms for the two events, respectively. The corresponding best repaired solutions are obtained within 69.032 ms and 33.599 ms. As shown in Fig. 3, during execution, agent 0 breaks down at $t = 15\,\text{s}$ and is removed from the available team. The planner then updates the current execution state and continues the remaining task allocation with the surviving agents. A more severe failure occurs at $t = 30\,\text{s}$, where Agent 20 also becomes unavailable. After this event, the online adaptation module re-identifies the current planning state and resumes the branch-and-bound search from the corresponding node. The unfinished subtasks are then reassigned to the remaining 23 agents. As shown in the timeline figure, no subsequent task segment is assigned to the failed agents, while the remaining agents continue or take over tasks such as p_1, p_5, and p_6. The replanning process preserves the task ordering and collaboration constraints throughout execution. In particular, tasks that are still feasible with the current collaborators continue execution, whereas tasks affected by failed agents are reassigned and executed by other compatible agents. The online search first obtains a feasible solution with cost 71, and further improves it to 62. Despite the two agent failures, all required tasks are eventually completed at approximately $62\,\text{s}$, demonstrating that the proposed adaptation method can maintain feasibility and improve the execution plan under unexpected team-size reductions.

XII. PHYSICAL EXPERIMENTS

XIII. CONCLUSION

In this work, a novel on-the-fly planning framework has been proposed for temporal-logic task allocation in heterogeneous multi-robot systems. The framework performs task allocation during score-guided NBA construction and uses GBA-level pruning to obtain an executable solution early. A branch-and-bound procedure further improves the solution. In addition, an online replanning strategy reassigns unfinished tasks when robots become unavailable. Theoretical analysis establishes completeness of our algorithm. Numerical and physical experiments validate the efficiency, solution quality, scalability, and adaptability of the proposed framework.

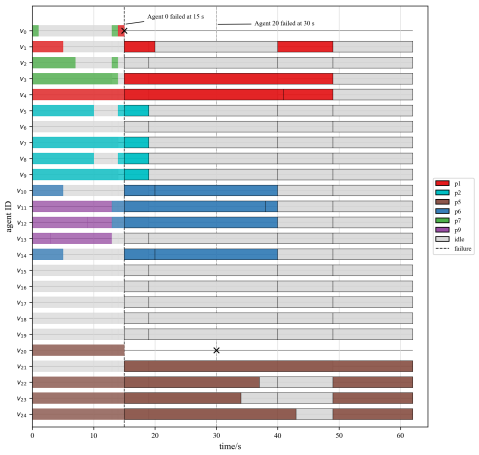

Fig. 3. Gantt graph of planning.

ACKNOWLEDGMENTS

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.

REFERENCES

[1] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, "Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints," *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.

[2] B. Li and H. Ma, "Double-deck multi-agent pickup and delivery: Multi-robot rearrangement in large-scale warehouses," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3701–3708, 2023.

[3] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, p. 111377, 2024.

[4] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT Press, 2008.

[5] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Decomposition of finite LTL specifications for efficient multi-agent planning," in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.

[6] L. Li, Z. Zhou, Z. Chen, H. Wang, Z. Kan, and J. Qin, "A formal framework for reactive heterogeneous multirobot task allocation in uncertain semantic environments," *IEEE Transactions on Cybernetics*, pp. 1–14, 2026.

[7] X. Luo and M. M. Zavlanos, "Temporal logic task allocation in heterogeneous multirobot systems," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3602–3621, 2022.

[8] K. Leahy, Z. Serlin, C.-I. Vasile, A. Schoer, A. M. Jones, R. Tron, and C. Belta, "Scalable and robust algorithms for task-based coordination from high-level specifications (scratches)," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2516–2535, 2022.

[9] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[10] Y. Kantaros and M. M. Zavlanos, "Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.

[11] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.

[12] P. Gastin and D. Oddoux, "Fast ltl to büchi automata translation," in *International Conference on Computer Aided Verification*, 2001.

[13] Z. Chen, Z. Zhou, S. Wang, J. Li, and Z. Kan, "Fast temporal logic mission planning of multiple robots: A planning decision tree approach," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6146–6153, 2024.

[14] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.

[15] M. Guo and D. V. Dimarogonas, "Multi-agent plan reconfiguration under local LTL specifications," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.

[16] P. Yu and D. V. Dimarogonas, "Distributed motion coordination for multirobot systems under ltl specifications," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 1047–1062, 2022.

[17] S. L. Smith, J. Tůmová, C. Belta, and D. Rus, "Optimal path planning under temporal logic constraints," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.

[18] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.

[19] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Multi-objective search for optimal multi-robot planning with finite ltl specifications and resource constraints," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 768–774.

[20] S. Karaman and E. Frazzoli, "Vehicle routing with linear temporal logic specifications: Applications to multi-uav mission planning," in *AIAA guidance, navigation and control conference and exhibit*, 2008, p. 6838.

[21] L. Li, Z. Chen, H. Wang, and Z. Kan, "Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities," *IEEE Transactions on Automation Science and Engineering*, vol. 22, no. 000, pp. 14 030–14 047, 2025.

[22] X. Luo and C. Liu, "Simultaneous task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Transactions on Robotics*, 2025.
[23] Z. Liu, M. Guo, W. Bao, and Z. Li, "Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products," *Research*, vol. 7, p. 0337, 2024.
[24] X. Luo, S. Xu, R. Liu, and C. Liu, "Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Robotics and Automation Letters*, vol. 9, no. 8, pp. 7182–7189, 2024.